

Stock Exchange Transaction Management

Using Doubly Linked List in C

 P.KARTHEEK & N.YASWANTH

The Problem We Solve

1

The Challenge of Dynamic Transactions

In a stock exchange environment, **large numbers of buy/sell transactions** are generated continuously. The number of transactions is not fixed, memory must be allocated dynamically, and frequent updates are required.

2

Why Static Structures Fail

Traditional static data structures are not suitable for this domain. They cannot adapt to unpredictable transaction volumes, lack the flexibility for real-time insertions and deletions, and waste memory by pre-allocating fixed capacity.

3

Our Objective

Design and implement a **menu-driven system in C** that efficiently manages stock transactions using dynamic memory allocation — supporting CRUD operations, sorting, and file storage.

Data Structures Used

struct Transaction

Each node stores: **Transaction ID**, Stock Name, Quantity, Price, and bidirectional pointers (`prev`, `next`). This struct encapsulates all the data needed for a single stock transaction record.

Doubly Linked List

Each node represents one transaction and carries a **pointer to both the previous and next node**. This enables efficient insertion, deletion, and traversal in both forward and reverse directions with dynamic memory allocation.

Why Doubly Linked List?

Efficient insertion and deletion at any position without shifting elements. Supports traversal in both directions. Memory is allocated dynamically using `malloc` and freed using `free()` — no wasted capacity.

Algorithm Explanation



Add Transaction

Create a new node using `malloc`. Read transaction details from user. Insert node at the end of the list. Update `prev` and `next` pointers accordingly.



Delete Transaction

Input transaction ID and traverse the list to find the node. Adjust pointers: `prev->next = next` and `next->prev = prev`. Free memory using `free()`.



Update & Search

Update: Search for transaction by ID, then modify fields (stock name, quantity, price). **Search:** Traverse list, compare ID, and display transaction if found.



Sort & File Handling

Sort by Price: Use nested loops to compare prices and swap node data. **Save/Load:** Open file in write/read mode, traverse list, and write or read each transaction record dynamically.

Features of the System

The Stock Exchange Transaction Management System is a fully menu-driven application built in C that covers the complete lifecycle of transaction management with robust memory handling.



CRUD Operations

- Create — Add new transactions
- Read — Display & Search records
- Update — Modify transaction fields
- Delete — Remove transactions



Memory & Traversal

- Dynamic memory allocation (malloc, free)
- Forward and reverse traversal
- Sorting transactions by price
- No memory leaks



File Handling

- Save to transactions.txt
- Load transactions from file
- Data persistence across sessions
- Dynamic node creation on load

Compilation & Project Files

1

Save the Source File

Save the program as `stock_exchange.c` in your working directory. Open your terminal or command prompt.

2

Compile Using GCC

Run: `gcc stock_exchange.c -o stock_exchange` — this produces the executable binary using the GCC compiler.

3

Execute the Program

Run: `./stock_exchange` — the menu-driven interface will launch, presenting all available transaction operations.



Project Files: `stock_exchange.c` — contains the full program source.
`transactions.txt` — stores transaction records permanently for data persistence.

Project Conclusion

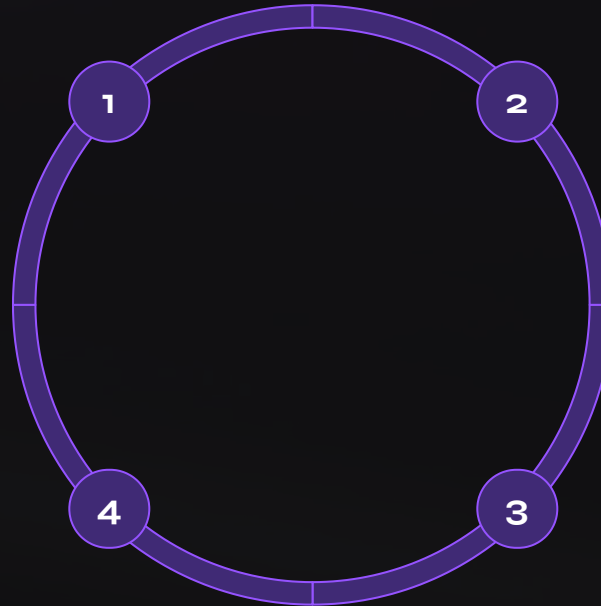
The Stock Exchange Transaction Management System successfully demonstrates the use of dynamic data structures in C. By using a doubly linked list, the system efficiently handles real-time transaction operations.

Dynamic Memory

Efficient use of `malloc` and `free()` with no memory leaks ensures scalable, real-world suitability.

Modular Design

Proper memory management and modular programming highlight the path toward developing scalable, maintainable systems in C.



CRUD Operations

Insertion, deletion, updating, and searching are all handled efficiently via doubly linked list traversal.

File Persistence

File handling enhances the system with data persistence, making it more practical and closer to real-world applications.

This project highlights the importance of proper memory management and modular programming in developing scalable systems.